

A 1.2.1-2 Representing Data



A 1.2.1 Describe the principal methods of representing data. -A

1.2.2 Explain how binary is used to store data.

Everything a computer holds, a number, a sentence, a photograph, a song, is stored as a pattern of binary digits. This page covers two linked ideas: how whole numbers are written in binary and hexadecimal and converted between those systems and decimal, and how binary encoding is then used to store every other kind of data. The first half is mechanical, and you should be able to do the conversions by hand. The second half is about what those 0s and 1s come to mean once we agree on an encoding. We work through the number systems and the conversions first, then take each data type the syllabus names, integers, characters and strings, images, audio and video, in turn.

Last Updated: 13 june 2026

Key Questions

A1.2.1

- What are the principal number systems used to represent integers, and how do binary and hexadecimal work?
- How do you convert whole numbers between binary, hexadecimal and decimal, in every direction?

A1.2.2

- What does it mean to say that all data is stored as binary?
- How are integers, characters and strings, images, audio and video each encoded in binary, and how does the encoding affect the space they take?

IB COMMAND TERM NOTE

This page carries two command terms, and they ask for different things.

Describe (A1.2.1) means give a detailed account. For the number systems, that means stating clearly what binary and hexadecimal representation are and carrying out the conversions accurately, showing your working. Marks come from correct method and correct answers, not from commentary around them.

Explain (A1.2.2) means give a detailed account including reasons. Saying "text is stored in binary" earns little. You need to say *how*: a character is assigned a numerical code, and that code is stored as bits. You should also be able to say *why* a given piece of data takes the space it does. Expect questions phrased as "explain how...", and answer with the mechanism, not just the outcome.

THINK FIRST

A light switch has two positions, on and off. Line up three switches side by side. How many different patterns can the three of them make together? Hold that number in mind as you read the next section.

Why computers work in binary

A computer is built from billions of tiny switches called transistors, each either on or off. Telling two states apart is easy and reliable: a voltage is either high or low. Building hardware that could reliably distinguish ten different voltage levels, one for each decimal digit, would be far harder and far more error-prone, so machines settle on two states.

Every value a computer holds is therefore a pattern of these two states, written as 0 and 1. A single one of these digits is a **bit** (a *binary* digit). The real question is not why binary, but how two symbols can stand for numbers, text, pictures and sound. That is what the rest of the page answers. (Three switches, by the way, give eight patterns: 2 to the power 3.)

The three number systems

All three systems use **place value**: the position of a digit decides what it is worth, and each position is worth a fixed multiple of the one to its right.

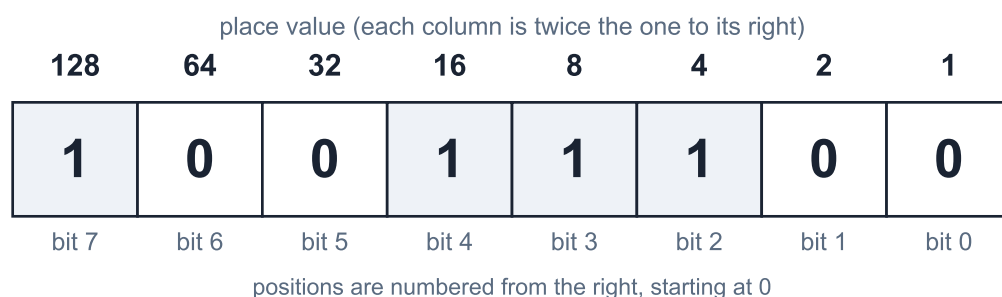
Decimal (base-10) is the system you grew up with. It uses the digits 0 to 9, and each position is worth ten times the one to its right: units, tens, hundreds, thousands. We use it because we have ten fingers, not because it suits computers.

Binary (base-2) uses only 0 and 1. Each position is worth twice the one to its right: 1, 2, 4, 8, 16 and so on. This is the system the machine actually stores.

Hexadecimal (base-16) uses the digits 0 to 9 and then the letters A to F to stand for the values ten to fifteen. Each position is worth sixteen times the one to its right. Hexadecimal exists for one reason: a single hex digit corresponds to exactly four bits, so it is a compact, human-readable shorthand for long binary strings. It is not a separate kind of number, and the computer does not store anything "in hex".

Denary	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
Hexadecimal	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E

The diagram below shows place value at work inside a single byte (eight bits).



A bit set to 1 adds its column value. The shaded columns are the 1s here:

$$128 + 16 + 8 + 4 = 156$$

Figure 1. Positional notation in a byte. Each column has a fixed value, and the bits set to 1 are added to give the denary number. Here 10011100 represents 156.

Converting between number systems

The syllabus asks you to convert whole numbers between all three systems, in both directions. (These conversions are for integers; fractional binary is not part of this learning statement, so we keep to whole numbers throughout.)

WORKED EXAMPLE

Denary to binary, by place value. Convert 156 to binary.

Write the binary place values from the right until you pass your number: 128, 64, 32, 16, 8, 4, 2, 1.

Work from the largest. Does 128 fit into 156? Yes, so put a 1 under 128. Remainder: $156 - 128 = 28$.

Does 64 fit into 28? No: 0. Does 32 fit into 28? No: 0.

Does 16 fit into 28? Yes: 1. Remainder 12. Does 8 fit into 12? Yes: 1. Remainder 4. Does 4 fit into 4? Yes: 1. Remainder 0.

Nothing is left, so the last two columns are 0.

128	64	32	16	8	4	2	1
1	0	0	1	1	1	0	0

So $156 = 10011100$.

WORKED EXAMPLE

Denary to binary, by repeated division. Same answer, different route. Divide repeatedly by 2 and record each remainder, then read the remainders from bottom to top.

Division	Result	Remainder
$156 \div 2$	78	0
$78 \div 2$	39	0
$39 \div 2$	19	1
$19 \div 2$	9	1
$9 \div 2$	4	1
$4 \div 2$	2	0
$2 \div 2$	1	0
$1 \div 2$	0	1

Reading upwards gives 10011100, which matches. Use whichever method you find more reliable under exam pressure; both are acceptable.

WORKED EXAMPLE

Binary to denary. Convert 10101101.

Write the place values above the bits and add the columns that hold a 1.

128	64	32	16	8	4	2	1
1	0	1	0	1	1	0	1

$$128 + 32 + 8 + 4 + 1 = 173.$$

WORKED EXAMPLE

Binary and hexadecimal. Because one hex digit is exactly four bits, you convert by splitting into groups of four.

Hex to binary: convert each digit to its four-bit code. 3FD4 becomes 0011 1111 1101 0100, so 3FD4 = 0011111111010100.

Binary to hex: split into fours from the right, padding the left group with zeros if needed, then convert each group. Take 1100111010011. Grouped and padded: 0001 1001 1101 0011, which reads as 1, 9, D, 3. So 1100111010011 = **19D3**.

EXAM TIP

When you convert binary to hexadecimal, always group the bits in fours starting from the right, and pad the leftmost group with leading zeros if it is short. Grouping from the left is the single most common way to arrive at the wrong answer.

WORKED EXAMPLE

Denary and hexadecimal. The cleanest route uses repeated division by 16.

Denary to hex: convert 2004. $2004 \div 16 = 125$ remainder 4. $125 \div 16 = 7$ remainder 13, and 13 is D. $7 \div 16 = 0$ remainder 7. Read the remainders upwards: 7, D, 4. So 2004 = **7D4**.

Hex to denary: convert 3A8. Multiply each digit by its place value and add: $(3 \times 256) + (10 \times 16) + (8 \times 1) = 768 + 160 + 8 = \mathbf{936}$.

COMMON EXAM MISTAKE

Students routinely treat binary, denary and hexadecimal as if they store different things. They do not. 11001110, 206 and CE are three ways of writing one value, and a question that asks you to convert is asking you to rewrite the same number, not to change it. The second

frequent error is reading a binary number from the wrong end: place value runs right to left, with bit 0 worth 1.

Storing data in binary

Everything above this point was about writing numbers. This section is about the syllabus's harder demand: how binary comes to hold text, pictures, sound and moving images.

The fundamentals: bits, fixed widths, and meaning

A few terms describe groups of bits.

Name	Size in bits
bit	1
nibble	4
byte	8
word	a larger processing group, commonly 16, 32 or 64

Two points follow from this, and both matter for the "impact on storage and retrieval" part of the learning statement. First, storage is usually allocated in fixed widths. A 32-bit integer occupies 32 bits whether it holds the value 3 or 3 billion, which makes storage predictable but means small values can waste space. Second, a pattern of bits has no meaning on its own. The byte 01000001 is the number 65, the letter A, or one channel of a colour, depending entirely on the encoding the program agrees to use. Retrieving data correctly therefore means knowing both where the bits are and how they were encoded. Choose the wrong encoding on the way out and you get nonsense, which is why a text file opened as an image shows noise.

Integers

What it is: a whole number, positive, negative or zero.

How it is stored: directly as binary, in a fixed-width field of 8, 16, 32 or 64 bits. The width sets the range of values that fit.

Example: in Java, an *int* is given 32 bits, which holds values from $-2,147,483,648$ to $2,147,483,647$. Ask for a value outside that range and it will not fit in the field.

Characters and strings

What it is: a character is a single symbol, a letter, digit or punctuation mark. A string is a sequence of characters.

How it is stored: each character is assigned a numerical code by an encoding standard, and that code is stored as bits. A string is then stored as the codes of its characters, one after another, so its size depends on the encoding and the number of characters.

Example: under ASCII, the string "Hello" is the codes 72, 101, 108, 108, 111, which in hexadecimal are 48 65 6C 6C 6F. The single character 'A' is code 65, or 41 in hexadecimal.

ASCII, designed in the 1960s, used only enough codes for 128 characters, which suited English but left no room for other scripts. **Unicode** was created to fix that: it assigns a unique number to every character in every writing system, so a single document can mix English, Arabic and emoji without clashing. Its most common encoding on the web, UTF-8, uses between one and four bytes per character and keeps the original ASCII codes for ordinary English text, which is why old text still opens correctly.

Images

What it is: a picture stored as a grid of pixels.

How it is stored: each pixel holds a colour, and in the common RGB model that colour is three values, the strengths of red, green and blue, each in one byte (0 to 255). Three bytes give 24 bits per pixel.

Example: pure red is (255, 0, 0), written FF0000 in the hexadecimal colour notation you will meet in web pages and design tools. With 8 bits per channel there are 256 levels each, giving $256 \times 256 \times 256 = 16,777,216$ possible colours.

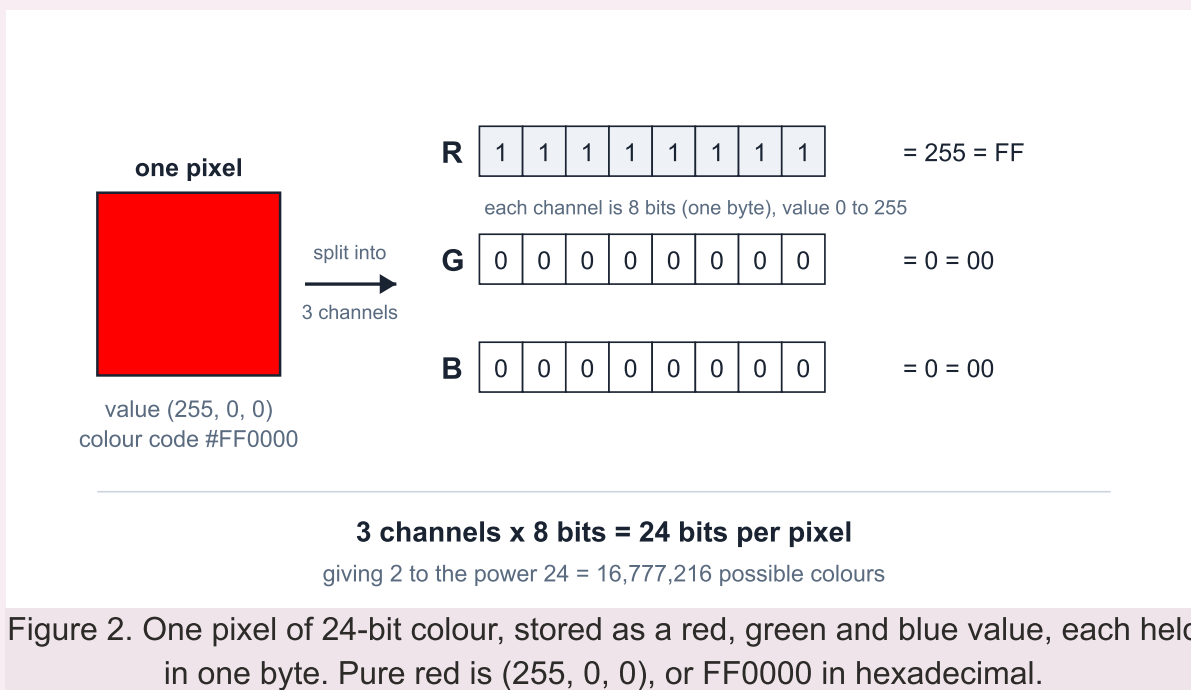


Figure 2. One pixel of 24-bit colour, stored as a red, green and blue value, each held in one byte. Pure red is (255, 0, 0), or FF0000 in hexadecimal.

Audio

What it is: a sound wave, which is continuous, captured as a stream of numbers.

How it is stored: the wave's amplitude is measured at regular intervals. How often you measure is the **sample rate**; how many bits each measurement is stored in is the **bit depth**. More bits per sample means finer steps and a more faithful recording.

Example: CD-quality audio samples 44,100 times a second at 16 bits per sample. A 16-bit sample can take 2 to the power $16 = 65,536$ different levels, fine enough that the steps are inaudible.

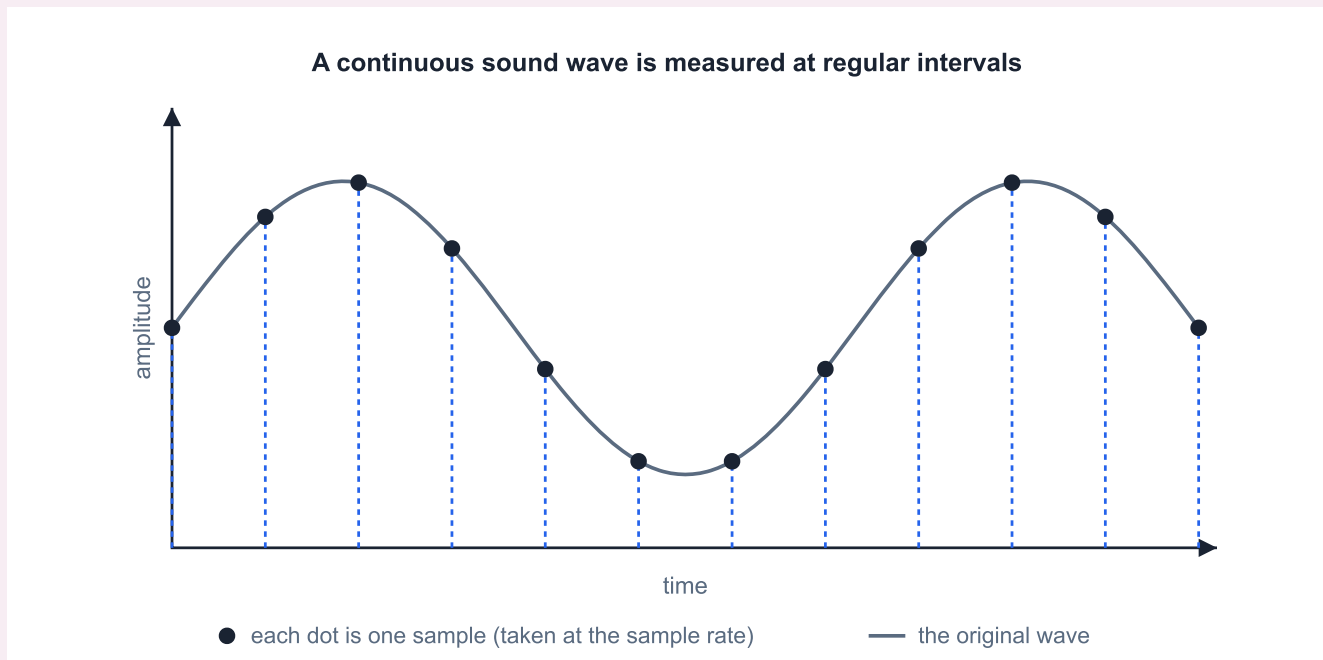


Figure 3. Sampling an audio signal. The continuous wave is measured at regular intervals (the sample rate), and each measured value is stored as a binary number whose size is the bit depth.

Video

What it is: moving pictures with synchronised sound.

How it is stored: a video is a sequence of still images, called frames, displayed fast enough to look continuous, usually alongside an audio track. Each frame is stored exactly as an image is, and the file holds many frames for every second of footage.

Example: a clip at 25 frames per second stores 25 full images for each second, plus the audio. Raw video is therefore enormous, which is why it is almost always compressed. (Compression itself is dealt with earlier in the syllabus.)

WORKED EXAMPLE

Why fidelity costs space. These two estimates show why uncompressed media files are large.

An image of 1920×1080 pixels at 24 bits (3 bytes) per pixel needs $1920 \times 1080 \times 3 = 6,220,800$ bytes, roughly 6 MB for a single still.

One minute of CD audio needs $44,100 \text{ samples} \times 16 \text{ bits} \times 2 \text{ channels} = 1,411,200 \text{ bits}$ per second. That is 176,400 bytes per second, so a minute is about 10.6 MB.

Now picture video at 25 of those images every second, and the reason compression exists becomes obvious.

SUMMARY REFERENCE

[Click here](#)

Number systems

System	Base	Symbols	Note
Binary	2	0, 1	what the machine stores
Denary	10	0 to 9	the human everyday system
Hexadecimal	16	0 to 9, A to F	shorthand for binary; 1 digit = 4 bits

Conversions

Conversion	Method
Denary to binary	place value subtraction, or repeated division by 2
Binary to denary	add the place values of the bits set to 1
Hex to binary	each hex digit becomes its 4-bit code
Binary to hex	group bits in fours from the right, pad left, convert each group

Conversion	Method
Denary to hex	repeated division by 16, remainders bottom to top
Hex to denary	multiply each digit by its place value and add

Storing data

Data type	How it is encoded	Typical size
Integer	direct binary, fixed width	8, 16, 32 or 64 bits
Character	a code from ASCII or Unicode	8 bits (ASCII); 1 to 4 bytes (UTF-8)
String	the codes of its characters in sequence	depends on length and encoding
Image	grid of pixels, each an RGB colour	24 bits per pixel (8 per channel)
Audio	amplitude sampled at regular intervals	bit depth per sample, e.g. 16 bits
Video	sequence of image frames plus audio	very large; usually compressed

TRY IT ON PAPER

Work these before the quiz. Model answers follow each question.

1. Convert the denary number 206 to binary and to hexadecimal, showing your working. [3 marks]

Model answer: $206 = 128 + 64 + 8 + 4 + 2$, so the binary is 11001110 **[1]** for correct method, **[1]** for 11001110. Grouping in fours from the right gives 1100 1110, which is C and E, so the hexadecimal is CE **[1]**.

2. Explain why hexadecimal is often used as a shorthand for binary. [2 marks]

Model answer: one hexadecimal digit represents exactly four bits [1], so long binary patterns can be written far more compactly and are easier for people to read and copy without error [1].

3. Explain how a colour image is stored in binary, referring to pixels and the RGB model. [4 marks]

Model answer: an image is stored as a grid of pixels [1]; each pixel stores a colour as three values for red, green and blue [1]; each of these channels is held in one byte, giving a value from 0 to 255 [1]; so each pixel uses 24 bits, and the whole image is the pixels stored one after another [1].

1

How many bits are there in one byte?

A. ☐ 1

B. ☐ 4

C. ☒ 8

D. ☐ 16

A byte is 8 bits. A nibble is 4 bits and a word is a larger group such as 16 bits, which is why those are tempting but wrong.

2

What is the binary number 10110 in denary?

A. ☐ 13

B. ☒ 22

C. ☐ 26

D. ☐ 44

The set bits are worth $16 + 4 + 2 = 22$. Option 13, comes from reading the bits in reverse order (01101), the most common slip on this kind of question.

3

What is the hexadecimal number 2F in denary?

A. ☐ 32

B. ☐ 35

C. ☒ 47

D. ☐ 215

$(2 \times 16) + 15 = 47$. Option 35, comes from treating the 2 as if it were worth ten rather than sixteen, that is, applying decimal place value to a hex number.

4

Using the RGB model with 8 bits per channel, how many bits store the colour of a single pixel?

A. ☐ 8

B. ☐ 16

C. ☒ 24

D. ☐ 32

Three channels of 8 bits each give 24 bits. 8 is a single channel only; 32 wrongly assumes a fourth byte.

5

A group of four bits is known as a:

A. ☐ bit

B. ☒ nibble

C. ☐ byte

D. ☐ word

Four bits make a nibble, which is also why one nibble maps to exactly one hexadecimal digit.

6

Why do computers represent data in binary rather than decimal?

A. ☐ Binary numbers are always shorter than decimal numbers

B. ☒ The hardware uses components with two reliable states, on and off

C. ☐ Decimal numbers are too large for a computer to store

D. ☐ Binary can represent more values than decimal

Two stable electrical states are easy to build and to tell apart. “Binary can represent more values than decimal” is a misconception: the system used does not change which values can be represented, only how they are written.

7

A 16-bit audio sample can represent how many distinct amplitude levels?

A. ☐ 16

B. ☐ 256

C. ☐ 32,768

D. ☒ 65,536

2 to the power 16 is 65,536. 256, is the figure for an 8-bit sample, the classic confusion between bit depth values.

8

Which statement about hexadecimal is correct?

- A. ☒ Each hexadecimal digit represents exactly four bits
- B. ☐ Hexadecimal can store more data than binary
- C. ☐ Hexadecimal is a base-8 number system
- D. ☐ Computers process instructions directly in hexadecimal

One hex digit maps to four bits, which is the whole reason it is a useful shorthand. “Hexadecimal can store more data than binary” is wrong because hexadecimal is only a notation, not a different store of data.

All materials on this website are for the exclusive use of teachers and students at subscribing schools for the period of their subscription. Any unauthorised copying or posting of materials on other websites is an infringement of our copyright and could result in your account being blocked and legal action being taken against you.

 **Report a problem**